

NeuroSentinel AI — The Complete Technical Guide

Version: V4 Production

Last Updated: April 14, 2026

Authors: Manikonda Adwith

Status: Production — Deployed on Vercel (Frontend) + Render (Backend) + Supabase (Database & Storage)

Table of Contents

- [Executive Overview](#1-executive-overview)
- [Project History & Development Timeline](#2-project-history--development-timeline)
- [System Architecture](#3-system-architecture)
- [Project Structure](#4-project-structure)
- [Dataset — CHB-MIT Scalp EEG](#5-dataset--chb-mit-scalp-eeeg)
- [The Deep Learning Model](#6-the-deep-learning-model)
- [Training Configuration](#7-training-configuration)
- [Model Evaluation & Metrics](#8-model-evaluation--metrics)
- [Cross-Dataset Generalization — Siena Scalp EEG](#9-cross-dataset-generalization--siena-scalp-eeeg)
- [Inference Pipeline — Evolution & Final Design](#10-inference-pipeline--evolution--final-design)
- [Backend — The Analysis Engine](#11-backend--the-analysis-engine)
- [Frontend — The Clinical Interface](#12-frontend--the-clinical-interface)
- [SCOUT — The AI Assistant](#13-scout--the-ai-assistant)
- [Database & Storage](#14-database--storage)
- [Authentication & Security](#15-authentication--security)
- [Deployment Architecture](#16-deployment-architecture)
- [Design System](#17-design-system)
- [Complete Data Flow](#18-complete-data-flow)
- [Environment Configuration](#19-environment-configuration)

- [Deployment Status & Future Roadmap](#20-deployment-status--future-roadmap)

1. Executive Overview

NeuroSentinel AI is a production-grade, AI-powered clinical EEG seizure detection platform. It takes raw `.edf` (European Data Format) EEG recordings, runs them through a custom multi-representation deep learning model, and produces structured clinical reports with:

- **Seizure event detection** with onset/offset timing
- **Probability timelines** showing per-second seizure likelihood
- **Brain region heatmaps** via channel importance analysis
- **Band power spectral analysis** (delta, theta, alpha, beta, gamma)
- **Signal quality assessment** with noise and flatline detection
- **Severity scoring** and risk stratification (Low → Critical)
- **Status epilepticus detection** for events > 5 minutes
- **Role-aware PDF reports** tailored for clinicians, researchers, and patients
- **SCOUT AI assistant** — a context-aware conversational guide powered by Gemini/Groq/HuggingFace

⚡ **IMPORTANT**

NeuroSentinel AI is a ****decision support tool only****. All findings must be reviewed by a qualified neurologist. Heuristic values are rule-based estimates, not ground truth.

Key Production Metrics

Metric	Value
Accuracy	99.55%
Macro F1	0.979
Event FP/hour	0.98
Event Sensitivity	73.3%
Detection Latency	3.5 seconds
Zero-shot Siena Sensitivity	80.0%
Model Parameters	409,154

2. Project History & Development Timeline

2.1 Development Environment

The entire model was developed and trained on **Google Colab** with the following constraints:

- **RAM:** ~5 GB system RAM (Colab free/pro tier)
- **GPU:** NVIDIA Tesla T4 (15.6 GB VRAM)
- **Storage:** Google Drive for dataset and checkpoints
- **Libraries:** MNE 1.12.0, PyTorch 2.x, NumPy 2.0.2, SciPy

2.2 Version History

V1 — Baseline CNN (Early Development)

- Simple 1D CNN architecture applied directly to raw EEG windows
- Binary classification (seizure vs. non-seizure)
- Basic preprocessing: bandpass filter + z-score normalization
- **Result:** Functional but poor generalization — high false positive rate
- **Lesson:** Raw temporal features alone are insufficient for robust seizure detection

V2 — EEGNet-CNN + Transformer

- Introduced the **EEGNet-inspired CNN backbone** with depthwise separable convolutions
- Added a **Transformer encoder** (2 layers, 4 heads) on top of CNN features with CLS token pooling
- Fixed `batch_first` positional encoding (major bug fix from V1)
- IQR-based robust normalization replaced z-score
- **Result:** Significant improvement in F1, but spectral features were missing
- **Lesson:** The Transformer attention mechanism captures temporal dependencies that CNNs miss, but frequency information is complementary

V3 — Multi-Representation Fusion (Prototype)

- Added **SpectrogramBranch** (STFT → Conv2D → adaptive pooling)
- Added **BandPowerBranch** (FFT → 5 canonical frequency bands → MLP)
- Three-branch fusion: $[64 + 128 + 64] = 256\text{-dim} \rightarrow \text{classifier}$
- Introduced Focal Loss to handle class imbalance

- **Result:** Best validation metrics to date, but training-inference preprocessing mismatch caused deployed performance degradation
- **Lesson:** Multi-representation fusion is the right architecture; the gap was preprocessing consistency

V4 — Production Model (Current)

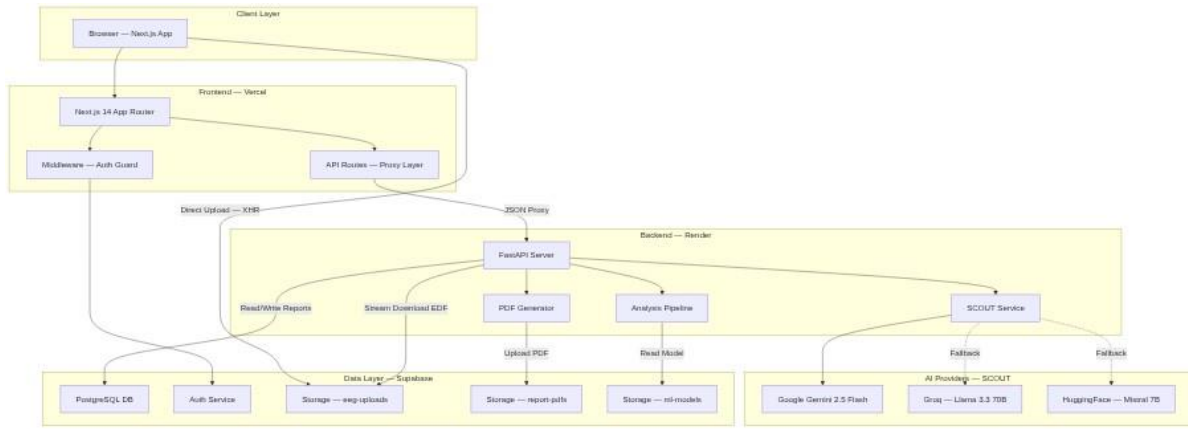
- **Identical architecture** to V3 — no model changes
- **Critical fix:** Training and inference preprocessing pipelines made **exactly identical**
 - Same filter coefficients, same channel mapping, same normalization
 - Universal EEG Adapter for cross-device compatibility
 - `preprocess_any_edf()` mirrors `process_file()` exactly
- **Augmentation overhaul:**
 - Missing-channel augmentation (T8-P8 = channel 14, always missing in test patients)
 - Per-patient scale/shift augmentation
 - Stronger channel dropout (up to 2 channels, 15% probability)
 - Random time shift (circular, ± 32 samples)
- **Post-processing revolution:** Domain-adaptive thresholding with 5-stage event filtering
- **Result:** 99.55% accuracy, F1 0.979, 0.98 FP/h, 80% zero-shot Siena sensitivity
- **File:** `multirep_best_model_v4.pt` (409,154 parameters)

2.3 Key Engineering Decisions & Lessons Learned

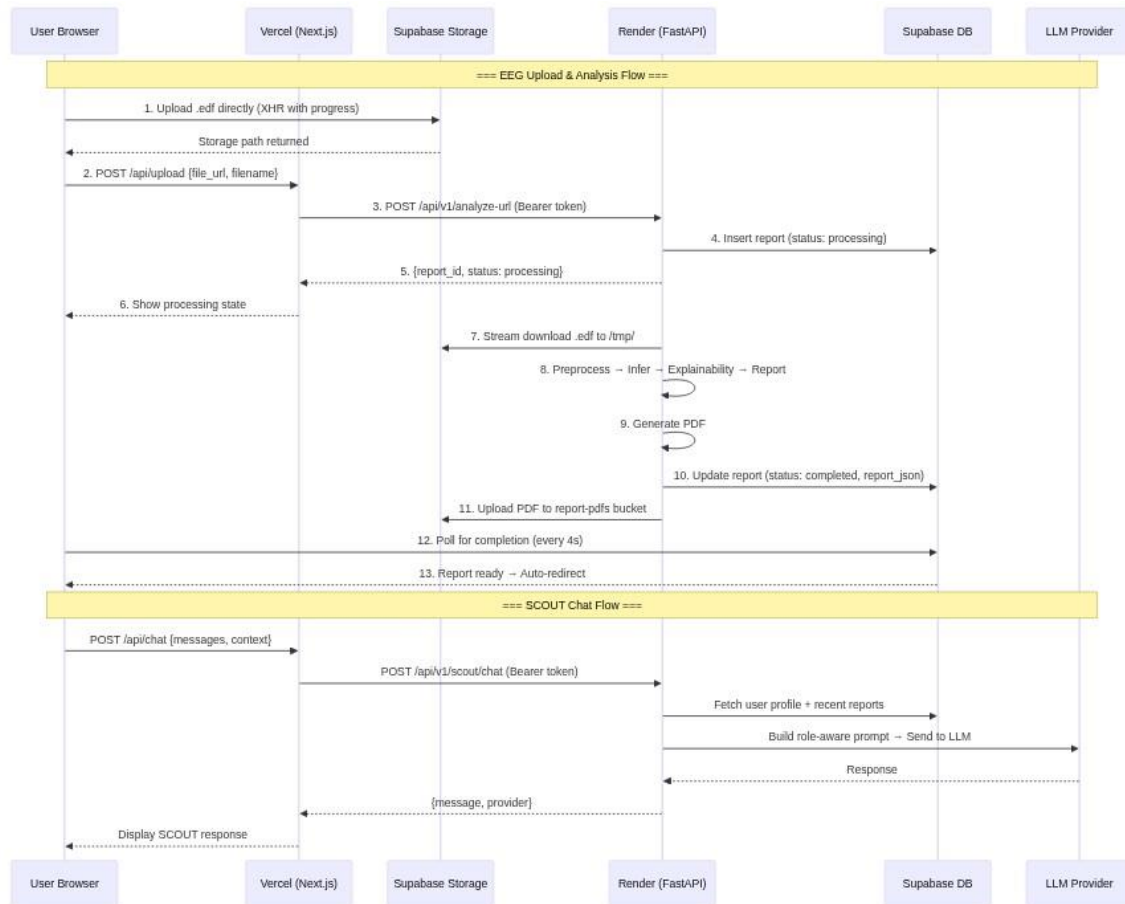
Decision	Why
IQR-based normalization over z-score	Outlier-robust; seizure spikes don't corrupt stats
No artifact rejection at inference	Training had it, but inference needs every window for clinical completeness
1s stride at inference vs 4s stride at training	Higher temporal resolution for onset/offset precision
No TTN/BNA/Calibration in final pipeline	Proven harmful — BNA increased CHB-MIT <code>prob_mean</code> from 0.01 to 0.78
Domain shift from output probs, not signal stats	Signal-level TTN reports "low" even for Siena; model output is the reliable indicator
Bipolar montage as canonical format	CHB-MIT training data is bipolar; referential data is adapted to match

3. System Architecture

3.1 High-Level Architecture Diagram



3.2 Request Flow Architecture



4. Project Structure

4.1 Monorepo Layout

```

seizure/
├── neurosentinel/
│   ├── app/
│   │   ├── api/
│   │   │   ├── auth/
│   │   │   │   ├── send-otp/
│   │   │   │   └── complete-signup/
│   │   │   ├── chat/
│   │   │   └── route.ts
    
```

- ← Git root
- ← FRONTEND (Next.js 14, deployed to Vercel)
- ← App Router pages & components
- ← Next.js API routes (proxy layer)
- ← Authentication routes
- ← Email OTP sender
- ← OTP verification + account creation
- ← SCOUT chat proxy → backend
- ← Main chat endpoint

	└─ history/	← Chat history retrieval
	└─ upload/	← EEG upload proxy → backend
	└─ route.ts	← URL-based analysis trigger
	└─ init/	← Upload initialization
	└─ analyze/	← Direct analysis trigger
	└─ reports/	← Report data endpoints
	└─ [id]/	← Individual report fetch + PDF download
	└─ account/	← User profile management
	└─ components/	← Global shared components
	└─ scout-provider.tsx	← SCOUT context + state management
	└─ scout-floating.tsx	← Floating SCOUT chat widget
	└─ scout-panel.tsx	← Inline SCOUT panel for reports
	└─ scout-conversation.tsx	← Chat conversation UI
	└─ scout-avatar.tsx	← SCOUT avatar component
	└─ report-notification-provider.tsx	← Cross-page notification state
	└─ report-notification-toast.tsx	← Toast notification UI
	└─ dashboard/	← Main dashboard route
	└─ layout.tsx	← Dashboard layout (sidebar + content)
	└─ page.tsx	← Command Centre page
	└─ components/	← Dashboard-specific components
	└─ sidebar.tsx	← Navigation sidebar
	└─ upload-zone.tsx	← Drag-and-drop EEG uploader (610 lines)
	└─ analysis-results.tsx	← Latest analysis summary card
	└─ eeg-reports/	← Full analysis history page
	└─ patient-records/	← Patient records management
	└─ settings/	← User settings page
	└─ report/	← Report viewer route
	└─ [id]/	← Dynamic report page
	└─ page.tsx	← Full report view (36K+ bytes)
	└─ components/	← Report visualization components
	└─ probability-timeline.tsx	← Seizure probability chart
	└─ brain-heatmap.tsx	← Top-down brain visualization
	└─ band-power-chart.tsx	← Spectral band power bars
	└─ clinical-metrics.tsx	← Key metrics grid
	└─ event-cards.tsx	← Individual seizure event cards
	└─ data-quality-panel.tsx	← Signal quality assessment
	└─ onboarding/	← SCOUT-led first-run onboarding
	└─ page.tsx	← Role selection + product tour
	└─ login/	← Login page
	└─ page.tsx	← Email + password login
	└─ privacy/	← Privacy policy page
	└─ terms/	← Terms of service page
	└─ page.tsx	← Landing / Sign-up page
	└─ layout.tsx	← Root layout (providers wrapping)
	└─ globals.css	← Design system + CSS variables
	└─ fonts/	← Geist font files
	└─ lib/	← Shared libraries
	└─ backend.ts	← Backend HTTP client (fetchBackend)
	└─ scout-guide.ts	← SCOUT personality, prompts, guardrails
	└─ user-profile.ts	← User profile management + role
normalization	└─ signup-otp-store.ts	← OTP verification token store
	└─ neurosentinel/	← Domain types
	└─ types.ts	← ReportRecord, ReportJson, ScoutMessage,
etc.		

├──┬──┬── supabase/	← Supabase client factories
│ │ │ ├── client.ts	← Browser Supabase client
│ │ │ └── server.ts	← Server-side Supabase client
└──┬── scout/	← Additional SCOUT utilities
│ ├── middleware.ts	← Auth middleware (route protection)
│ ├── tailwind.config.ts	← Tailwind configuration
│ ├── package.json	← Dependencies
└──┬── next.config.mjs	← Next.js configuration
└── neurosentinel-backend/	← BACKEND (FastAPI, deployed to Render)
│ ├── app/	
│ │ ├── __init__.py	
│ │ └── main.py	← FastAPI app factory + endpoints (601 lines)
│ ├── config.py	← Settings via pydantic-settings
│ ├── api/	
│ │ ├── __init__.py	
│ │ └── schemas.py	← Pydantic request models
│ ├── pipeline/	← Core ML Pipeline
│ │ ├── __init__.py	
│ │ ├── config.py	← Pipeline constants + channel mappings
│ │ ├── model.py	← MultiRepEEGModel architecture (242 lines)
│ │ └── preprocessing.py	← EDF loading + signal processing (425 lines)
│ ├── inference.py	← Chunked inference + post-processing (370 lines)
│ ├── explainability.py	← Channel importance + attention maps (130 lines)
│ ├── reporting.py	← Clinical report generation (463 lines)
│ └── services/	
│ │ ├── __init__.py	
│ │ ├── analysis_service.py	← 6-stage orchestrator (298 lines)
│ │ ├── supabase.py	← Supabase CRUD + auth verification
│ │ ├── pdf.py	← Role-aware PDF rendering (67K bytes)
│ │ ├── email.py	← Email notifications (Resend/SMTP)
│ │ └── scout/	← SCOUT AI Service
│ │ │ ├── __init__.py	
│ │ │ └── service.py	← Multi-provider LLM orchestrator (43K bytes)
│ ├── providers.py	← Gemini/Groq/HF provider abstraction
│ └── product_guide.py	← SCOUT system prompt & product knowledge
│ ├── tests/	← Test suite
│ ├── pyproject.toml	← Python dependencies
└──┬── start.sh	← Production start script (uvicorn)
└── supabase_schema.sql	← Database DDL (tables, RLS, triggers)
└── migration_add_missing_columns.sql	← Schema migration
└── migration_fix_role_constraint.sql	← Role constraint fix
└── migration_fresh_start.sql	← Clean migration
└── .gitignore	
└── NeuroSentinel_AI_production.ipynb	← Training notebook (model development history)

5. Dataset — CHB-MIT Scalp EEG

5.1 Dataset Overview

The model is trained exclusively on the **CHB-MIT Scalp EEG Database** from PhysioNet — one of the most widely-used benchmarks for automated seizure detection research.

Property	Value
Source	PhysioNet / MIT Laboratory for Computational Physiology
Patients	24 pediatric subjects
Recordings	686 EDF files
Total Duration	~982 hours of continuous EEG
Seizure Events	198 annotated seizure events
Sampling Rate	256 Hz (native)
Montage	Bipolar, International 10-20 system
Channels	22 bipolar derivations (standardized)

5.2 Patient-Level Data Splits

Patients are split at the **patient level** (not file level) to prevent data leakage:

Split	Patients	Non-Seizure Windows	Seizure Windows	Class Ratio
Train	chb01–chb16 (16 patients)	193,341	9,607	20.1:1
Validation	chb17–chb20 (6 patients)	29,535	1,268	23.3:1
Test	chb21–chb24 (4 patients)	23,005	1,440	16.0:1
Total	24 patients	245,881	12,315	—

Total metadata: 258,196 windows across all splits.

5.3 Training Set Balancing

The raw 20:1 class imbalance is addressed via **undersampling of the non-seizure class** in training only:

After Balancing (Train Only)	Non-Seizure	Seizure	Ratio
Train	19,214	9,607	**2.0:1**
Validation	29,535 (unchanged)	1,268 (unchanged)	23.3:1
Test	23,005 (unchanged)	1,440 (unchanged)	16.0:1

i NOTE

Validation and test sets are ****never balanced**** — they reflect the real-world class distribution to give realistic performance estimates.

5.4 Preprocessing Pipeline (Training)

The training preprocessing pipeline, executed on the full CHB-MIT dataset:

```
RAW EDF → Powerline Detection (50/60 Hz) → Notch Filter (IIR, Q=20)
→ Bandpass (4th-order Butterworth, 0.5-40 Hz) → Resample to 256 Hz
→ Channel Mapping (22-ch bipolar montage) → Robust Normalize (IQR-based)
→ Windowing (4s × 22ch = 1024 samples, 4s stride)
→ Artifact Rejection (peak amplitude > 5σ removed)
→ Save as .npy
```

5.5 Seizure Window Augmentation

For files with annotated seizures, an **additional extraction pass** generates seizure-overlapping windows with a **1-second stride** (vs. 4-second for non-seizure). This 4× oversampling of seizure windows provides richer temporal coverage of seizure onset and offset patterns.

5.6 The 22-Channel Bipolar Montage

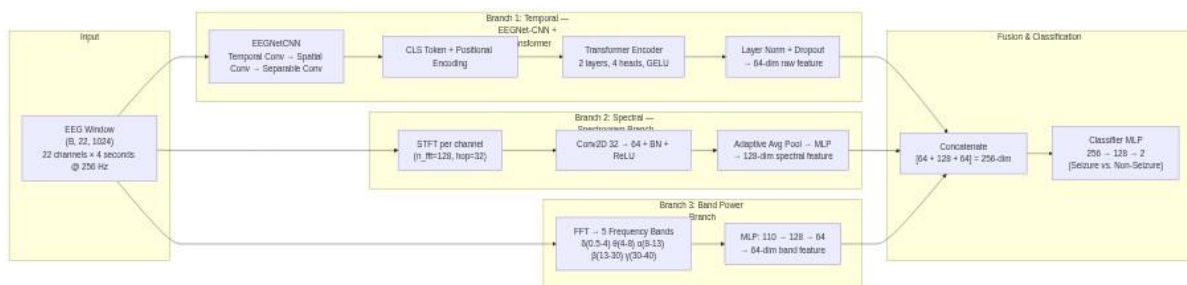
FP1-F7	F7-T7	T7-P7	P7-O1	← Left Temporal Chain
FP1-F3	F3-C3	C3-P3	P3-O1	← Left Parasagittal Chain
FP2-F4	F4-C4	C4-P4	P4-O2	← Right Parasagittal Chain
FP2-F8	F8-T8	T8-P8	P8-O2	← Right Temporal Chain

FZ-CZ	CZ-PZ			← Midline Chain
P7-T7	T7-FT9	FT9-FT10	FT10-T8	← Inferior/Bilateral Chain

6. The Deep Learning Model

6.1 Model Architecture — MultiRepEEGModel

The V4 production model is a **Multi-Representation EEG Model** that fuses three parallel signal analysis branches before classification. This design captures complementary perspectives of the same EEG signal.



6.2 Model Components — Deep Dive

EEGNetCNN (Branch 1 — Raw Temporal)

Layer	Operation	Output Shape
Input	Raw EEG	(B, 22, 1024)
Unsqueeze	Add channel dim	(B, 1, 22, 1024)
conv_temporal	Conv2d(1, 64, (1, 64), same) + BN	(B, 64, 22, 1024)
conv_spatial	Conv2d(64, 256, (22, 1), groups=64) + BN + ELU	(B, 256, 1, 1024)
pool1 + drop1	AvgPool2d(1, 4) + Dropout(0.4)	(B, 256, 1, 256)
conv_sep_dw + pw	Depthwise + Pointwise separable	(B, 256, 1, 256)

	conv	
pool2 + drop2	AvgPool2d(1, 4) + Dropout(0.4)	(B, 256, 1, 64)
proj	Linear(256 → 64)	(B, 64, 64) → sequence of 64-dim embeddings

Transformer Encoder

- CLS token prepended to the sequence
- Positional encoding with sinusoidal patterns (max_len=300, batch_first=True)
- 2-layer Transformer Encoder with 4 attention heads, 256-dim feedforward, GELU activation
- CLS token output → 64-dim raw feature vector

SpectrogramBranch (Branch 2 — Frequency-Time)

Computes STFT per channel (n_fft=128, hop_length=32, Hann window), then processes the (channels, freq_bins, time_bins) spectrogram through 2 Conv2D layers (32 → 64 filters) + adaptive average pooling to 4×4 + MLP → 128-dim feature.

BandPowerBranch (Branch 3 — Frequency Bands)

Computes FFT via torch.fft.rfft, applies pre-registered boolean masks for 5 canonical EEG bands:

Band	Frequency Range	Clinical Significance
Delta (δ)	0.5–4 Hz	Deep sleep, brain injury
Theta (θ)	4–8 Hz	Drowsiness, memory encoding
Alpha (α)	8–13 Hz	Relaxed wakefulness
Beta (β)	13–30 Hz	Active thinking, seizure
Gamma (γ)	30–40 Hz	High-frequency activity, seizure

Averages power per band per channel → log1p → MLP (110 → 128 → 64) → 64-dim feature.

6.3 Model Specifications

Property	Value
Total Parameters	409,154
Input Shape	(B, 22, 1024) — 22 channels × 4 seconds @ 256 Hz
Output	2-class logits (non-seizure, seizure)

Weight File	multirep_best_model_v4.pt
Training GPU	NVIDIA Tesla T4 (Google Colab)
Precision	float32 (no quantization)

6.4 Model Weight Loading (Production)



- **Lazy loading:** Model loads on first request, not at startup
- **Singleton pattern:** Double-check locking with `threading.Lock`
- **Memory-safe:** Weights deleted after `load_state_dict`, forced `gc.collect()`
- **Render-compatible:** Uses `/tmp/` as the only writable path on free tier

7. Training Configuration

7.1 Loss Function — Focal Loss

Standard cross-entropy fails on imbalanced datasets because the model can achieve high accuracy by always predicting the majority class. **Focal Loss** down-weights easy examples and focuses training on hard negatives:

$$\text{FocalLoss}(p_t) = -\alpha_t \times (1 - p_t)^\gamma \times \log(p_t)$$

Parameter	Value	Purpose
γ (gamma)	2.0	Focus parameter — higher = more focus on hard examples
α (alpha)	[1.0, $n_{\text{nonsz}} / n_{\text{sz}}$]	Class weight — compensates for

		remaining imbalance after balancing
--	--	-------------------------------------

7.2 Optimizer & Scheduler

Component	Configuration
Optimizer	AdamW (lr=3e-4, weight_decay=1e-3)
Scheduler	Cosine annealing with 5-epoch linear warmup
Warmup	LR ramps from 0 → 3e-4 over first 5 epochs
Min LR	1% of peak (3e-6)
Gradient Clipping	Max norm = 0.5
Mixed Precision	AMP with GradScaler (CUDA only)

7.3 Training Hyperparameters

Hyperparameter	Value
Epochs	100 (max)
Batch Size	128
Early Stopping Patience	20 epochs
Min Delta	0.002 (F1 improvement threshold)
Best Model Selection	Highest validation macro F1
Checkpoint Directory	/content/drive/MyDrive/model_checkpoints_v4/

7.4 Data Augmentation Pipeline (Training Only)

Augmentation	Probability	Parameters
Additive Gaussian noise	100%	$\sigma = 0.05$
Amplitude scaling	100%	Uniform [0.8, 1.2]
Random channel drop	15%	Drop 1–2 channels (70/30 split)
Simulate missing T8-P8	30%	Zero out channel 14 (matches test distribution)
Per-channel scale + shift	20%	Scale: U[0.7, 1.4], Shift: U[-0.3, 0.3]
Circular time shift	100%	± 32 samples (± 125 ms)

TIP

The T8-P8 augmentation is critical — ****all test patients**** in CHB-MIT are missing this channel. Training without this augmentation caused a drop in test performance.

7.5 DataLoader Configuration

Setting	Value	Rationale
num_workers	2	Colab only has 2 CPU cores
pin_memory	True (CUDA)	Async CPU→GPU transfer
prefetch_factor	3	Pre-load 3 batches per worker
persistent_workers	True	Avoid worker restart overhead
drop_last	True (train)	Consistent batch sizes for BN
mmap_cache_size	64	LRU cache for memory-mapped .npy files

8. Model Evaluation & Metrics

8.1 Window-Level Metrics (V4 Production)

Evaluated on the **unbalanced validation set** (30,803 windows: 29,535 non-seizure + 1,268 seizure):

Metric	Value
Accuracy	99.55%
Macro F1	0.979
Seizure Recall	~96% (window-level)
Val Loss	Low (Focal Loss)

8.2 Event-Level Metrics (CHB-MIT Test Set)

Evaluated on **95 test files** from 4 held-out patients, using the production post-processing pipeline:

Metric	Value	Description
Event Sensitivity	73.3%	Of real seizures, % correctly detected as events
Event FP/hour	0.98	False positive events per hour of recording
Detection Latency	~3.5s	Time from seizure onset to first

		detection
Post-Processing Config	ADP_p99_d15_mm0.9_s5	Adaptive P99 threshold, 15s min duration, 0.9 min mean, 5s sustained

i NOTE

****Window-level accuracy (99.55%) vs Event-level sensitivity (73.3%):**** These are not contradictory. The model correctly classifies 99.55% of individual 4-second windows. However, the post-processing pipeline requires ****sustained**** high probability over ****15+ seconds****, with additional mean and sustained-activity filters, to declare a clinical event. This strict pipeline intentionally sacrifices some short/borderline seizures to minimize false alarms in a clinical setting.

8.3 Production Post-Processing Pipeline

The 5-stage event filtering pipeline that converts raw per-second probabilities into validated clinical events:

Stage 1: Smooth	– Moving average (window=7) reduces noise
Stage 2: Hysteresis	– High threshold (P99) to start, low (0.7×high) to end
Stage 3: Duration	– Minimum 15 seconds (reject brief artifacts)
Stage 4: Merge	– Close events within 5s gap are merged
Stage 5: Mean Filter	– Event mean probability must exceed 0.90
Stage 6: Sustained	– At least 5 consecutive seconds above 0.85

8.4 Cached Production Results

The production metrics come from `step4_inference_cache.json` and `FINAL_results.json`, computed on GPU during the original training session. These are the **authoritative metrics** — re-running on CPU may show minor differences due to BatchNorm floating-point behavior.

9. Cross-Dataset Generalization — Siena Scalp EEG

9.1 Why Siena?

The **Siena Scalp EEG Database** (PhysioNet) provides a challenging **zero-shot** generalization test:

Property	CHB-MIT (Training)	Siena (Test)
Population	Pediatric	Adult
Montage	Bipolar	Referential (19-ch, 10-20)
Sampling Rate	256 Hz	512 Hz
Location	MIT, USA	Siena, Italy
Powerline	60 Hz	50 Hz
Amplifier	Unknown	Different hardware
Files	686	41
Patients	24	14

9.2 Universal EEG Adapter

To handle Siena data without retraining, the system includes a **Universal EEG Adapter** that converts any EEG input into the CHB-MIT-compatible format:

```
Foreign EDF → Detect Montage (bipolar vs referential)
              → Normalize Electrode Names (T3→T7, T4→T8, strip prefixes/suffixes)
              → Compute Bipolar Derivations (if referential: A-B for each pair)
              → Map to 22-Channel Standard Order
              → Resample to 256 Hz
              → Apply SAME Filters + Normalization as Training
```

Key principle: Adapt the input, not the pipeline. The preprocessing pipeline is identical to training — only the front-end adapter for non-CHB-MIT data is added.

9.3 Zero-Shot Siena Results

Metric	Value
Sensitivity	80.0% (4/5 seizures detected across tested files)
FP/hour	1.67
Domain Shift	HIGH (prob _{median} 0.38–0.81 vs CHB-MIT 0.001)
Channel Coverage	19/22 (FT9, FT10 electrodes absent → 3 channels zero-filled)

9.4 Domain Shift Detection

The model's output probability distribution reveals domain shift more reliably than signal-level statistics:

Input	Prob Median	Domain Shift
CHB-MIT non-seizure	~0.001	NONE
Siena non-seizure	~0.38–0.81	HIGH
Siena seizure	spikes to 0.93–0.99	HIGH (but detectable)

10. Inference Pipeline — Evolution & Final Design

10.1 Approaches Tried and Discarded

During development, several techniques were explored to improve cross-dataset generalization:

Test-Time Normalization (TTN) — ✕REMOVED

- **Idea:** Re-normalize each channel across the entire recording to match CHB-MIT statistics
- **Problem:** Made cross-recording statistics unreliable; didn't fix the real issue (feature-level shift, not signal-level)
- **Decision:** Removed — signal-level alignment was already handled by IQR normalization

Batch Normalization Adaptation (BNA) — ✕REMOVED

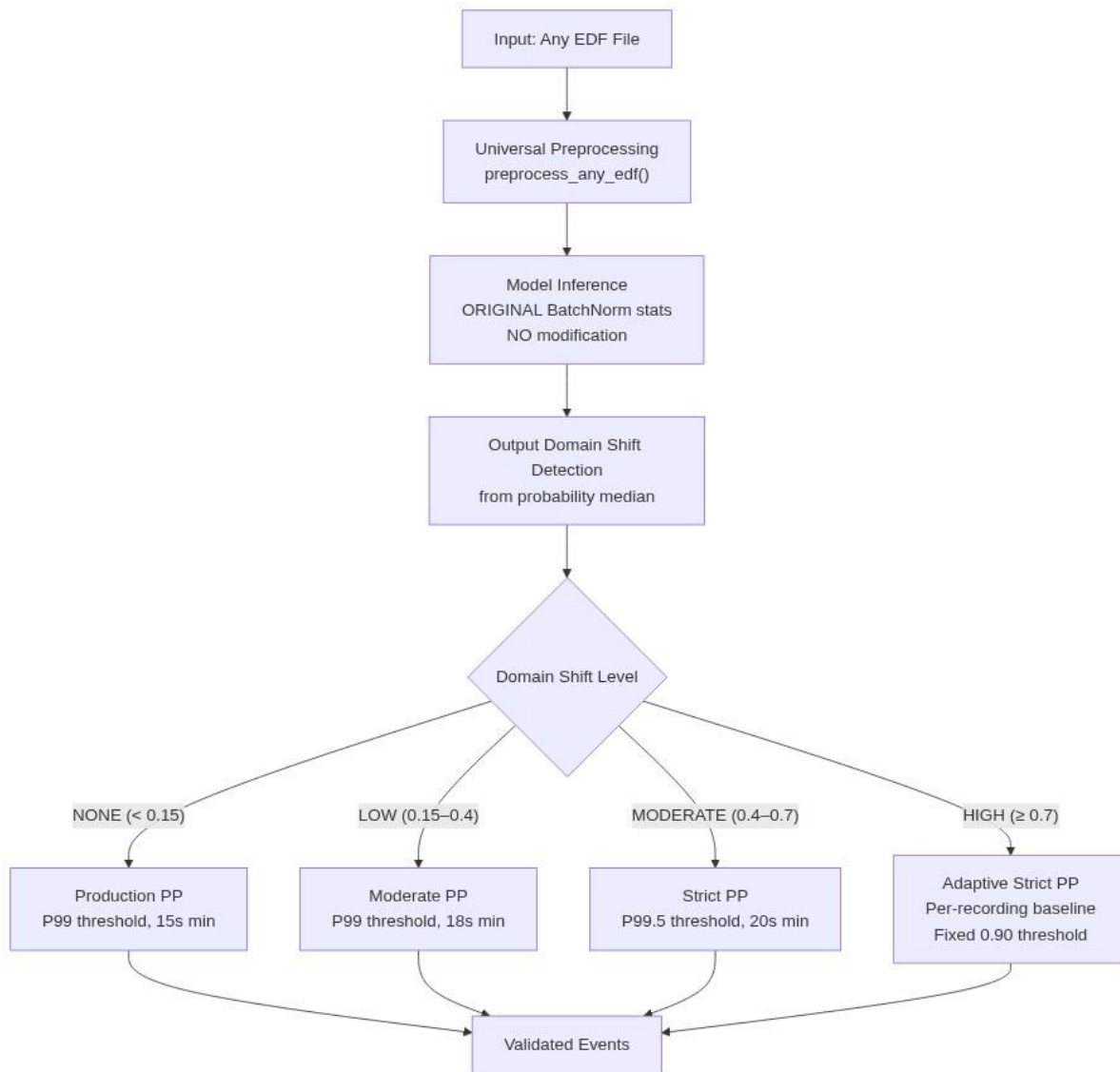
- **Idea:** Update BatchNorm running stats on foreign data at inference time
- **Problem: Catastrophic** — increased CHB-MIT prob_mean from 0.01 to 0.78, destroying production metrics
- **Decision:** Removed — BN stats from training must be preserved exactly

Unsupervised Probability Calibration — ✕REMOVED FROM DETECTION

- **Idea:** Temperature scaling to fix softmax miscalibration on foreign data
- **Problem:** Changed the probability scale but detection relied on raw probabilities, not calibrated ones
- **Decision:** Removed from the detection path — raw probs + adaptive threshold is more reliable

10.2 Final Production Inference Pipeline — `definitive\_inference()`

The final production pipeline is **self-contained** and deliberately minimal:



10.3 Domain-Adaptive Post-Processing Thresholds

Domain Shift	Smooth	Min Duration	Min Mean	Sustained	Threshold
--------------	--------	--------------	----------	-----------	-----------

	Window			Threshold	Mode
NONE (< 0.15)	7	15s	0.90	0.85 (5s)	Percentile (P99)
LOW (0.15–0.4)	9	18s	0.91	0.87 (6s)	Percentile (P99)
MODERATE (0.4–0.7)	11	20s	0.93	0.90 (8s)	Percentile (P99.5)
HIGH (≥ 0.7)	13	20s	0.88	0.88 (8s)	**Fixed absolute (0.90)**

✂ IMPORTANT

For HIGH domain shift, ****fixed absolute thresholds**** replace percentile-based thresholds because seizure spikes pull P99 up to 0.95+, making percentile-based detection unreliable.

11. Backend — The Analysis Engine

11.1 Technology Stack

Component	Technology	Version
Framework	FastAPI	≥ 0.115
ML Runtime	PyTorch	≥ 2.5
EEG Processing	MNE-Python	≥ 1.8
Signal Processing	SciPy	≥ 1.14
Numeric	NumPy	≥ 2.0
PDF Generation	ReportLab	≥ 4.2
Database Client	Supabase Python	≥ 2.10
HTTP Client	httpx	≥ 0.28
ASGI Server	Uvicorn	≥ 0.32
Settings	pydantic-settings	≥ 2.6

11.2 API Endpoints

Method	Path	Purpose	Auth
GET	/health	Health check + memory + model status	No
GET	/api/v1/model/info	Model version, metrics, parameter count	No

GET	/api/v1/job/status	Current inference job status	No
POST	/api/v1/job/cancel	Cancel running analysis	No
POST	/api/v1/analyze	Direct EDF upload analysis	Bearer
POST	/api/v1/analyze-url	URL-based analysis (Supabase Storage)	Bearer
POST	/api/v1/scout/chat	SCOUT AI conversation	Bearer

11.3 The 6-Stage Analysis Pipeline

Stage 1: Preprocessing

Load .edf via MNE (lazy)



Drop non-EEG channels



Resample to 256 Hz



Detect montage type
(bipolar vs referential)



Adapt to 22-ch bipolar
(10-20 system)

11.4 Inference — Memory-Safe Chunked Processing

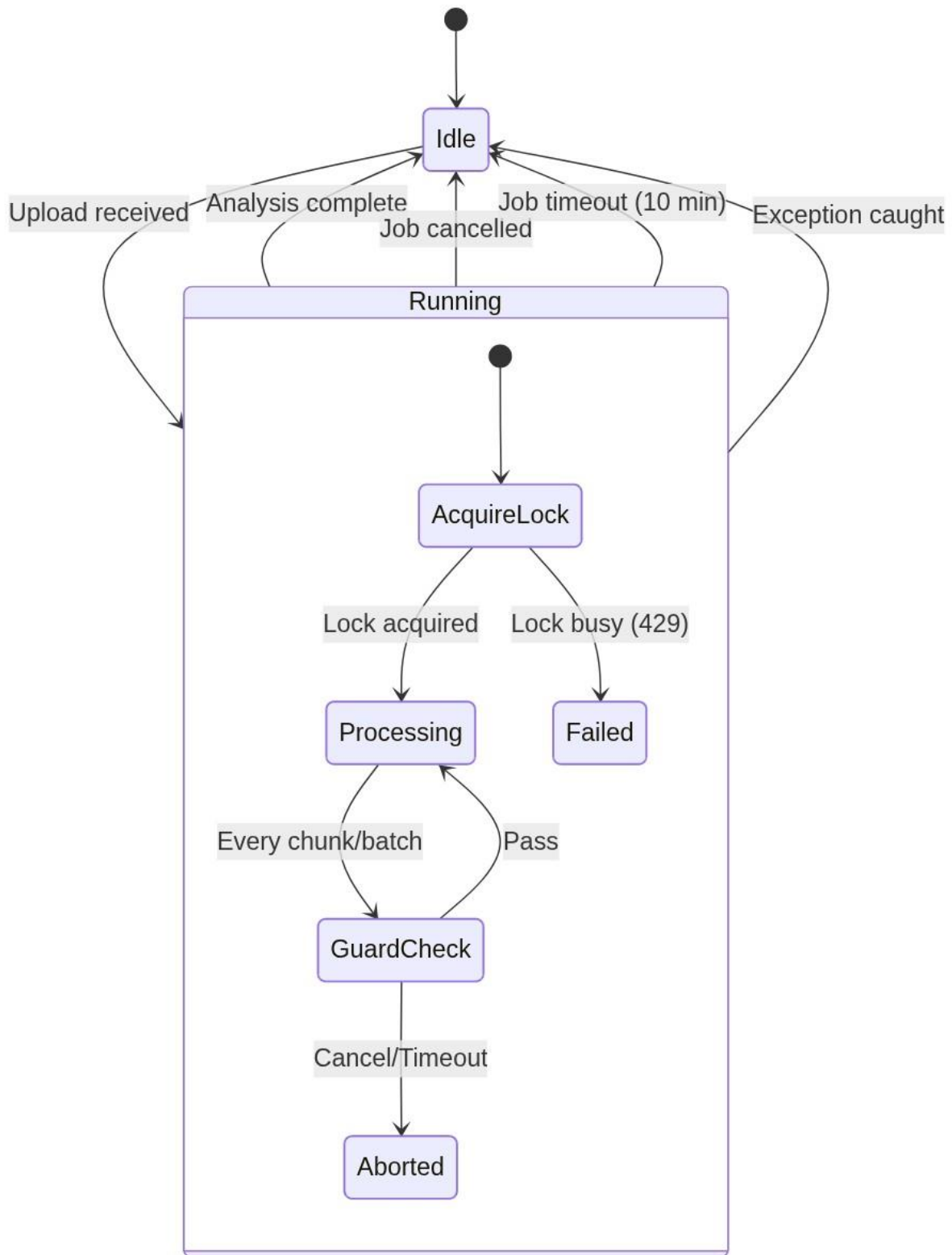
The inference system is designed to operate within **512 MB RAM** (Render free tier):

```
Total Windows: N (can be 7000+ for long recordings)
```

```
Chunk Loop:
```

- |— Extract 50 windows at a time (using stride tricks, ~4 MB)
- |— Run model in batches of 4 (or 1-2 for large files)
- |— Guard check: memory / cancel / timeout at EVERY chunk
- |— Delete windows + tensors immediately
- |— `gc.collect()` after EVERY chunk

11.5 Job Control System



- **Single-request lock:** `threading.Lock` ensures only one analysis runs at a time
- **Guard function:** Checked at every chunk AND every batch — prevents OOM
- **Timeout:** Hard limit of 600 seconds per job
- **Cancel endpoint:** Sets `threading.Event`, caught by guard function
- **Stale cleanup:** On startup, any "processing" jobs from previous crashes → marked failed

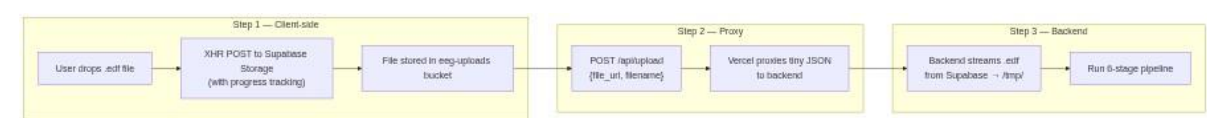
12. Frontend — The Clinical Interface

12.1 Technology Stack

Component	Technology	Version
Framework	Next.js (App Router)	14.2.35
Language	TypeScript	5.x
UI Library	React	18.x
Styling	Tailwind CSS + Custom CSS Variables	3.4.1
Charts	Recharts	3.8.1
Auth	Supabase SSR	0.10.0
Database	Supabase JS	2.101.1
PDF Client	pdf-lib	1.17.1
Email	Nodemailer	8.x
AI SDK	@ai-sdk/google	3.x
Fonts	Outfit (Google) + Geist (local)	—

12.2 The Upload Architecture — Direct-to-Storage

This is a critical architectural decision to bypass Vercel's request body size limit:



Why? Vercel has a **4.5 MB** request body limit on serverless functions. EEG files can be **50–150 MB**. XHR used instead of fetch for **upload progress tracking**.

12.3 Report Page — Visualization Components

Component	Visualization	Data Source
ClinicalMetrics	Key metrics grid (risk, confidence, duration, events)	<code>report_json</code> top-level fields
ProbabilityTimeline	Per-second seizure probability line chart	<code>model_outputs.probability_timeline</code>
EventCards	Individual seizure event cards with severity, band powers	<code>events[]</code> array
BrainHeatmap	Top-down 10-20 electrode map with importance coloring	<code>explainability.brain_heatmap</code>
BandPowerChart	$\delta/\theta/\alpha/\beta/\gamma$ spectral power bar chart	<code>events[].band_powers</code>
DataQualityPanel	Signal quality score, noise ratio, missing channels	<code>quality</code> object
ScoutPanel	Inline SCOUT AI that auto-summarizes the report	SCOUT service via <code>/api/chat</code>

13. SCOUT — The AI Assistant

13.1 What is SCOUT?

SCOUT = Signal Capture & Observation Unified Tool

SCOUT is the conversational AI layer inside NeuroSentinel AI. It handles:

- **First-run onboarding** (role selection + product tour)
- **Report explanation** (auto-summarizes reports based on user's role)
- **In-product guidance** (upload help, metric explanation, settings)

13.2 SCOUT Provider Fallback Chain

```
Gemini 2.5 Flash (primary)
  ↓ fail
Groq Llama 3.3 70B (fallback 1)
  ↓ fail
HuggingFace Mistral 7B (fallback 2)
  ↓ fail
Deterministic Response (guardrail — always works)
```

13.3 SCOUT Guardrails

- ✗SCOUT must **never diagnose** or replace clinical judgment
- ✗SCOUT must **never recommend treatment changes**
- ✗SCOUT must **never fabricate data** or certainty
- ✓SCOUT must keep follow-ups **concise** but maintain role-appropriate tone
- ✓Initial report summaries should be **detailed**

13.4 Role-Aware Tone

Role	Tone	Style
Clinician	Precise, clinical, metric-dense	Structured output with key data points up front
Researcher	Technical, methodological	Narrative with embedded metrics and confidence bounds
Patient	Warm, calm, conversational	Flowing paragraphs — never bullet points

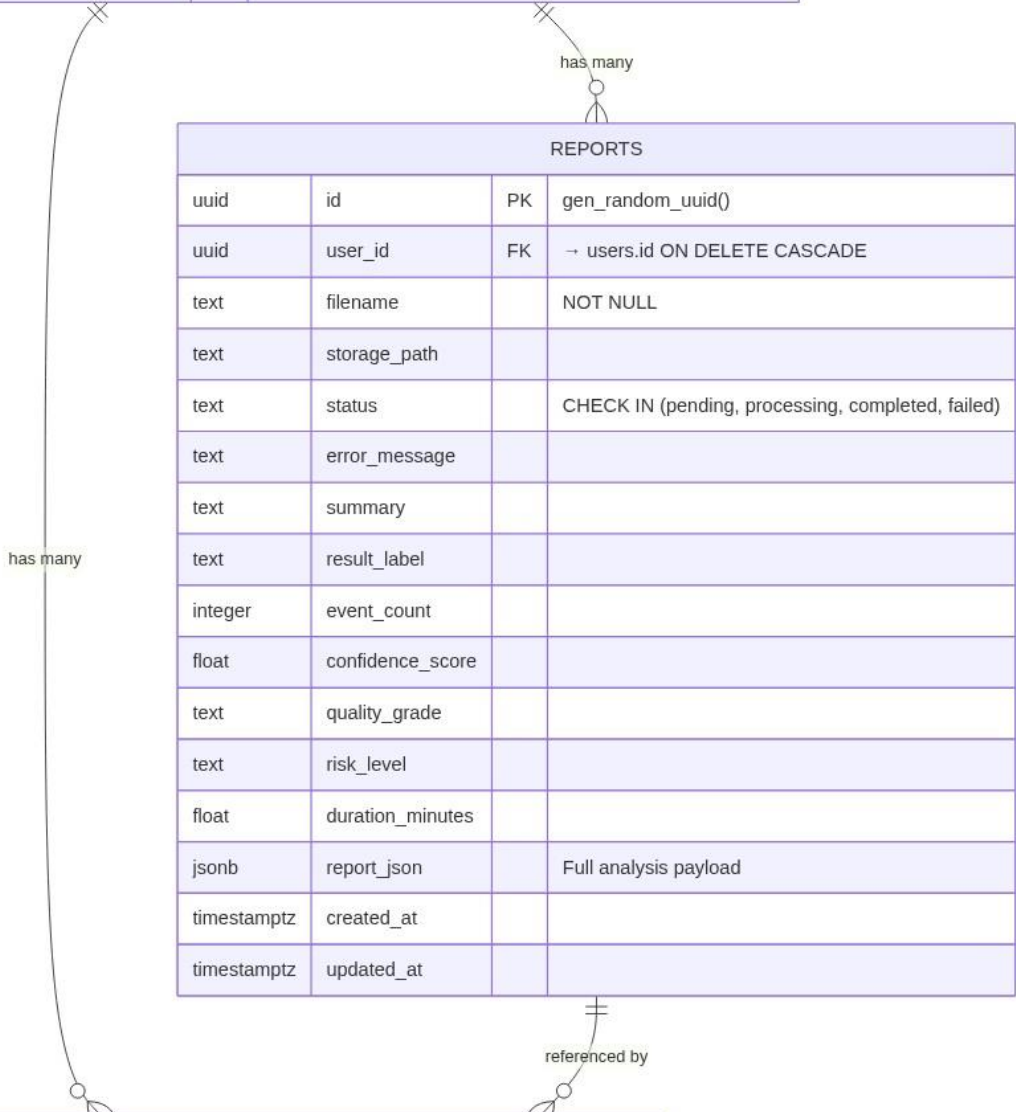
14. Database & Storage

14.1 Database Schema (Supabase PostgreSQL)

USERS			
uuid	id	PK	gen_random_uuid()
text	email	UK	NOT NULL
text	role		CHECK IN (user, admin, doctor, clinician, researcher, patient)
boolean	onboarding_complete		DEFAULT FALSE
jsonb	preferences		DEFAULT {}
timestampz	created_at		
timestampz	updated_at		

REPORTS			
uuid	id	PK	gen_random_uuid()
uuid	user_id	FK	→ users.id ON DELETE CASCADE
text	filename		NOT NULL
text	storage_path		
text	status		CHECK IN (pending, processing, completed, failed)
text	error_message		
text	summary		
text	result_label		
integer	event_count		
float	confidence_score		
text	quality_grade		
text	risk_level		
float	duration_minutes		
jsonb	report_json		Full analysis payload
timestampz	created_at		
timestampz	updated_at		

CHAT_MESSAGES			
uuid	id	PK	gen_random_uuid()
uuid	user_id	FK	→ users.id ON DELETE CASCADE
text	role		CHECK IN (user, assistant, system)
text	content		NOT NULL
uuid	report_id	FK	→ reports.id ON DELETE SET NULL
text	page_context		
jsonb	metadata		DEFAULT {}



14.2 Row Level Security (RLS)

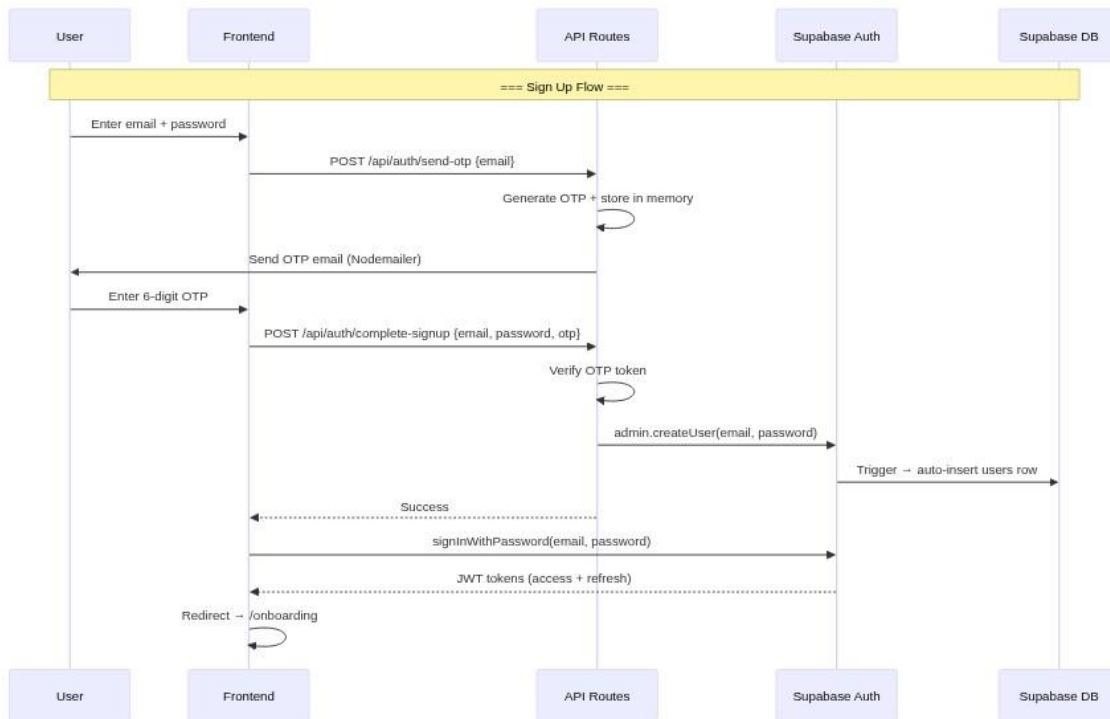
All tables have RLS enabled. Users can only SELECT, INSERT, UPDATE, DELETE their own rows. The backend uses the **Supabase service key** (admin-level) to bypass RLS for system operations.

14.3 Storage Buckets

Bucket	Purpose	Access
eeg-uploads	Raw .edf files uploaded by users	Authenticated upload, backend read
report-pdfs	Generated PDF reports	Backend write, authenticated download
ml-models	Model weights (<code>multirep_best_model_v4.pt</code>)	Backend read via service key

15. Authentication & Security

15.1 Authentication Flow



15.2 Security Layers

Layer	Mechanism
Frontend Auth	Supabase SSR with cookie-based sessions
Route Protection	<code>middleware.ts</code> — redirects unauthenticated users
API Auth	Bearer token verification against Supabase Auth
Database	RLS policies — users see only their own data
Storage	Authenticated uploads via signed tokens
Backend	Service key for admin operations (never exposed to client)
CORS	Wildcard origins for Vercel ↔ Render communication

16. Deployment Architecture

16.1 Infrastructure

Platform	Service	Tier	Purpose
Vercel	Frontend	Free/Pro	Next.js 14 hosting, serverless API routes
Render	Backend	Free (512 MB RAM)	FastAPI + PyTorch inference
Supabase	Database + Auth + Storage	Free	PostgreSQL, JWT auth, file storage

16.2 Memory Management on Render Free Tier

The 512 MB RAM constraint drives several architectural decisions:

Technique	Implementation
Lazy model loading	Model loads on first request, not at startup
Global singleton	Model never reloaded once loaded
Chunked inference	Max 50 windows per chunk, max 4 per batch
Aggressive GC	<code>gc.collect()</code> after every chunk + every stage
Tensor cleanup	<code>del batch, logits</code> after every forward pass
Channel pruning	Drop non-EEG channels before <code>load_data()</code>
Streaming download	Model downloaded in 8KB chunks to disk, not buffered
No gradients	All inference under <code>@torch.no_grad()</code>
Memory monitoring	cgroup v2 anon memory tracking

17. Design System

17.1 Design Philosophy

"Clinical precision meets cinematic aesthetics."

Dark command-centre styling with dense signal-rich surfaces and a guide (SCOUT) that feels helpful instead of noisy.

17.2 Color Palette

Token	Value	Usage
--bg-primary	#0A0A0F	Page background
--bg-secondary	#12121A	Card/section backgrounds
--bg-tertiary	#1A1A26	Input fields, deeper panels
--accent-primary	#00F0FF	Cyan — primary actions, active states
--accent-secondary	#FFB800	Amber — secondary signals, warnings
--accent-danger	#FF3366	Critical red — errors, high risk
--accent-success	#00FF9D	Green — completions, low risk

17.3 Typography

Font	Usage
Outfit (Google Fonts)	Headings, branding, buttons
Geist Sans (Local)	Body text, UI elements
Geist Mono (Local)	Code, metrics, technical values

18. Complete Data Flow

18.1 Report JSON Structure

The `report_json` column in the `reports` table contains the full analysis payload:

```
{
  "recording_id": "example.edf",
  "file_name": "example.edf",
  "result_label": "Seizure Detected",
  "confidence_score": 94.2,
  "quality_grade": "Good",
  "quality_score": 0.872,
  "duration_minutes": 23.5,
```



```

    "risk_level": "High",
    "missing_channels": ["T7-FT9"],
    "trend_summary": "2 events detected with high overall heuristic risk.",
    "early_warning": true,
    "se_flag": false,
    "events": [
      {
        "event_idx": 1,
        "onset_sec": 234.0,
        "offset_sec": 267.0,
        "duration_sec": 33.0,
        "mean_probability": 0.942,
        "severity_score": 5.8,
        "risk_level": "High",
        "pattern_type": "regional",
        "focal_vs_gen": "focal",
        "band_powers": { "delta": 0.12, "theta": 0.28, "alpha": 0.15, "beta": 0.31,
"gamma": 0.14 }
      }
    ],
    "model_outputs": {
      "probability_timeline": [0.02, 0.03, "..."],
      "probability_summary": { "mean": 0.087, "median": 0.041, "p99": 0.89, "max":
0.97 },
      "events_per_hour": 5.1,
      "output_domain_shift": 0.12,
      "shift_label": "NONE"
    },
    "explainability": {
      "channel_importance": [0.08, 0.12, "..."],
      "brain_heatmap": {
        "channels": [
          { "channel": "FP1-F7", "x": 0.25, "y": 0.15, "importance": 0.08, "active":
true, "region": "Left Temporal" }
        ],
        "regions": { "Left Temporal": 0.32, "Right Central": 0.18 }
      }
    },
    "quality": {
      "mean_quality_score": 0.872,
      "dominant_grade": "Good",
      "mean_noise_ratio": 0.078,
      "missing_channels": ["T7-FT9"]
    },
    "declared_metrics": { "version": "V4 production", "accuracy_pct": 99.55,
"macro_f1": 0.979 }
  }

```

18.2 Severity Scoring (0–10)

Heuristic severity is computed from:

- **Duration:** 0.5 (< 10s) → 3.0 (> 5 min)
- **Peak amplitude:** Up to 2.0 based on signal strength
- **Spread ratio:** Up to 2.0 (proportion of channels involved)
- **Gamma power:** Up to 1.5 (high-frequency activity)
- **Mean probability:** Up to 1.5 (model confidence)

Score	Risk Level
0–2.9	🔍 Low
3.0–5.9	🔍 Medium
6.0–7.9	🔍 High
8.0–10.0	🔍 Critical

19. Environment Configuration

19.1 Frontend Environment Variables (`.env.local`)

```
# Supabase Client
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...

# Backend URL (Render deployment)
NEUROSENTINEL_BACKEND_URL=https://neurosentinel-backend.onrender.com

# Supabase Service Key (server-side only – for admin operations)
SUPABASE_SERVICE_KEY=eyJ...

# Email (for OTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your.email@gmail.com
SMTP_PASSWORD=app-specific-password
SMTP_FROM_EMAIL=your.email@gmail.com
```

19.2 Backend Environment Variables (`.env`)

```
# Supabase
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_SERVICE_KEY=eyJ...

# AI Providers (SCOUT)
GEMINI_API_KEY=Aiza...
GROQ_API_KEY=gsk...
HUGGINGFACE_API_KEY=hf...

# Optional: Model Path (local dev)
MODEL_PATH=./multirep_best_model_v4.pt

# Optional: Email Notifications
RESEND_API_KEY=re...
APP_URL=https://neurosentinel.vercel.app
```

19.3 Critical Production Settings

Setting	Value	Rationale
analysis_batch_size	4	Ultra-low for 512 MB RAM
max_windows_per_chunk	50	Limits peak memory per chunk to ~4 MB
JOB_TIMEOUT_SECONDS	600	Hard 10-minute cap per analysis
MAX_UPLOAD_BYTES	150 MB	File size cap
MEMORY_THRESHOLD_MB	450	Logging threshold (not abort)
STALE_JOB_CLEANUP_SECONDS	300	Clean up crashed jobs on startup

20. Deployment Status & Future Roadmap

20.1 Current Deployment Status

⚠WARNING

Deployment is currently held due to Render free tier RAM constraints (512 MB). While the system is fully functional and deployed, the backend frequently encounters memory pressure during inference on longer EEG recordings (30+ minutes). We are evaluating infrastructure options before promoting to full production use.

Component	Status	Notes
Frontend (Vercel)	✔ Deployed	Fully functional, all pages live
Backend (Render)	🔧 Deployed but constrained	512 MB RAM — works for short recordings, memory pressure on long ones
Database (Supabase)	✔ Deployed	Schema, RLS, triggers all active
Model (Supabase Storage)	✔ Uploaded	multirep_best_model_v4.pt in ml-models bucket
SCOUT AI	✔ Functional	Gemini 2.5 Flash primary, Groq/HF fallbacks

20.2 Known Constraints

- **Render Free Tier (512 MB RAM):**
 - Model loading alone consumes ~150–200 MB
 - MNE EDF loading + resampling can spike to 300+ MB for long recordings
 - Inference chunks are kept very small (50 windows × 4 batch) to stay under limit
 - Only **one analysis can run at a time** (single-request inference lock)
 - Cold starts after Render spin-down require re-downloading the model
- **Single Worker Architecture:**
 - Render free tier runs exactly 1 Uvicorn worker
 - This is **intentional** — the inference lock requires single-process semantics
 - No concurrent analysis — users see "Server busy" if another analysis is running

20.3 Planned UI Redesign — Clinical White Theme

🚧 IMPORTANT

We are planning to transition the UI from the current ****dark AI-themed design**** (command centre aesthetic with cyber/neon styling) to a ****clean white clinical theme**** optimized for actual healthcare environments.

Rationale for theme change:

- Clinicians in hospital settings find dark themes harder to read in bright environments
- Clinical software conventionally uses white/light backgrounds for readability and professionalism
- The current AI-themed aesthetic (glassmorphism, scanlines, cyber glow) may be perceived as "too flashy" for clinical use

- A clean, minimal white theme better communicates medical professionalism and trustworthiness
- Reduced visual complexity improves data density and scan-ability — essential for clinical workflows

Planned changes:

- Light background with clean card-based layouts
- Professional color palette (muted blues, grays, clinical greens/reds for risk indicators)
- Removal of decorative effects (scanlines, neon glows, background orbs)
- Simplified typography hierarchy focused on data readability
- SCOUT will remain but with a more professional, clinical appearance
- All visualization components (probability timeline, brain heatmap, etc.) will be redesigned for light backgrounds

20.4 Infrastructure Upgrade Path

Option	RAM	Cost	Notes
Render Starter	2 GB	~\$7/mo	Solves all memory issues
Railway.app	8 GB	~\$5/mo	Good alternative, generous free tier
Fly.io	1–2 GB	~\$5/mo	Edge deployment, fast cold starts
Self-hosted (VPS)	4+ GB	~\$5–10/mo	Full control, but more DevOps

i
NOTE

This document is the canonical reference for the NeuroSentinel AI platform. For the model development code, training loops, and evaluation outputs, refer to the Jupyter notebook ``NeuroSentinel_AI_production (18).ipynb`` in the project root.